Student handbook: tuning classification models in scikit-learn

Oct 30, 2026 · @Swabhav Mentors

This handbook covers nine classification models in scikit-learn: which settings matter, what values to try, and ready-to-run code that finds the best values with grid search. Run the setup code in the first section once, then work through any algorithm section on its own. Practice exercises are at the end.

After working through it, you should be able to:

- name the two or three parameters that matter most for each algorithm
- build a `Pipeline` and a `param_grid`, and run `GridSearchCV`
- read the results and spot overfitting, poor scoring choices and data leakage

Getting started

Every algorithm below is tuned the same way: put the model in a `Pipeline`, list candidate values in a `param_grid`, and let `GridSearchCV` try every combination with 5-fold cross-validation. All code was run on scikit-learn 1.9.1 with the built-in breast cancer dataset (569 samples, 30 numeric features, 2 classes).

Hyperparameters are settings you choose before training, such as `C` in SVM or `max_depth` in a decision tree. The model does not learn them from the data, so we search for good values.

Two naming rules to remember:

- Inside a pipeline, a parameter is written as `<step name>__<parameter>` with a double underscore. `model__C` means "the `C` parameter of the step named `model`".
- The number of fits is the product of all list lengths times the number of folds. A grid of 5 × 3 × 2 values with `cv=5` means 30 × 5 = 150 fits.

Run this setup cell once. Every snippet below reuses `X_train`, `y_train` and `run_grid`.

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

def run_grid(pipe, param_grid):
    grid = GridSearchCV(pipe, param_grid, cv=5, scoring="accuracy",
n_jobs=-1)
    grid.fit(X_train, y_train)          # tries every combination
    print("Best parameters:", grid.best_params_)
    print(f"Best CV accuracy: {grid.best_score_:.3f}")
    print(f"Test accuracy:    {grid.score(X_test, y_test):.3f}")
    return grid
```

The scaler sits inside the pipeline on purpose. During cross-validation it is fitted only on the training folds, so no information leaks from the validation fold.

Quick comparison

If you tune only one or two parameters per algorithm, tune the ones in the last column.

Algorithm	scikit-learn class	Scale features?	Tune these first
Logistic Regression	`LogisticRegression`	Yes	`C`, `l1_ratio`
K-Nearest Neighbors	`KNeighborsClassifier`	Yes, essential	`n_neighbors`, `weights`
Naive Bayes (numeric)	`GaussianNB`	No	`var_smoothing`
Naive Bayes (text)	`MultinomialNB`	No	`alpha`

Algorithm	scikit-learn class	Scale features?	Tune these first
Support Vector Machine	`SVC`	Yes, essential	`C`, `kernel`, `gamma`
Decision Tree	`DecisionTreeClassifier`	No	`max_depth`, `min_samples_leaf`
Random Forest	`RandomForestClassifier`	No	`n_estimators`, `max_features`, `max_depth`
Gradient Boosting	`GradientBoostingClassifier`, `HistGradientBoostingClassifier`	No	`learning_rate`, `n_estimators` / `max_iter`, `max_depth`
AdaBoost	`AdaBoostClassifier`	No	`n_estimators`, `learning_rate`

Tree-based models split on one feature at a time, so the scale of a feature does not change their splits. Distance- and margin-based models (KNN, SVM) and regularized linear models do depend on scale.

Logistic Regression

`C` is the parameter that matters most: it controls how strongly the model is regularized. Small `C` means strong regularization and a simpler model; large `C` means weak regularization and a closer fit to the training data.

Parameter	Default	What it controls	Typical values to try
`C`	1.0	Inverse of regularization strength	0.01, 0.1, 1, 10, 100
`l1_ratio`	0.0	Type of regularization: 0 = L2, 1 = L1, in between = Elastic-Net	0, 1 (0.5 needs `solver="saga"`)

Parameter	Default	What it controls	Typical values to try
`solver`	`"lbfgs"`	Optimization algorithm	`"lbfgs"` (L2 only), `"liblinear"` (L1 or L2, small data), `"saga"` (all, large data)
`class_weight`	None	Extra weight for rare classes	None, `"balanced"`
`max_iter`	100	Iterations before stopping	Raise to 1000+ if you see a ConvergenceWarning

Version note: from scikit-learn 1.8, `penalty` is deprecated and will be removed in 1.10. Use `l1_ratio=0` instead of `penalty="l2"`, `l1_ratio=1` instead of `penalty="l1"`, and `C=np.inf` for no penalty. On older versions, write `penalty` in the grid instead of `l1_ratio`.

```
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(solver="liblinear", max_iter=1000)),
])

param_grid = {
    "model__C": [0.01, 0.1, 1, 10, 100],
    "model__l1_ratio": [0, 1],            # 0 = L2, 1 = L1
    "model__class_weight": [None, "balanced"],
}

lr_grid = run_grid(pipe, param_grid)       # 20 combinations x 5 folds = 100
fits
```

Result on the breast cancer data: best `C=0.1` with L2, 0.980 cross-validation accuracy, 0.982 test accuracy.

K-Nearest Neighbors

`n_neighbors` (k) is the parameter that matters most. Small k follows the training data closely and can overfit; large k gives a smoother boundary and can underfit. Always scale features, because KNN measures distances.

Parameter	Default	What it controls	Typical values to try
`n_neighbors`	5	How many neighbors vote	3, 5, 7, 9, 11, 15 (odd values avoid ties in binary problems)
`weights`	`"uniform"`	Whether closer neighbors count more	`"uniform"`, `"distance"`
`p`	2	Distance type for the Minkowski metric: 1 = Manhattan, 2 = Euclidean	1, 2
`metric`	`"minkowski"`	Distance function	`"minkowski"`, `"cosine"` (text or high-dimensional data)
`algorithm`	`"auto"`	How neighbors are searched; affects speed, not accuracy	Leave as `"auto"`

```
from sklearn.neighbors import KNeighborsClassifier

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", KNeighborsClassifier()),
])

param_grid = {
    "model__n_neighbors": [3, 5, 7, 9, 11, 15],
    "model__weights": ["uniform", "distance"],
    "model__p": [1, 2],                  # 1 = Manhattan, 2 = Euclidean
}

knn_grid = run_grid(pipe, param_grid)      # 24 combinations x 5 folds = 120
fits
```

Result on the breast cancer data: best k = 7, uniform weights, Euclidean distance, 0.971 cross-validation accuracy, 0.974 test accuracy.

Naive Bayes

Naive Bayes has very few parameters, and the main one is the smoothing strength. Pick the variant that matches your feature type first; tuning matters much less than that choice.

Variant	Use it for	Key parameters	Typical values to try
`GaussianNB`	Continuous numbers (marks, hours, measurements)	`var_smoothing` (default 1e-9): adds a small value to every variance for stability	`np.logspace(-11, -5, 7)`
`MultinomialNB`	Word counts (text)	`alpha` (default 1.0): add-alpha smoothing, the "+1" of Laplace smoothing; `fit_prior` (default True): learn class priors or use equal priors	`alpha`: 0.01, 0.1, 0.5, 1.0, 2.0
`BernoulliNB`	Yes/no features (word present or absent)	`alpha`; `binarize` (default 0.0): threshold that turns numbers into 0/1	`alpha` as above; `binarize`: 0.0, or None if data is already 0/1
`CategoricalNB`	Categories (Coaching = Yes/No)	`alpha`; `min_categories`: number of categories per feature, useful if a category is missing from training	`alpha` as above

Naive Bayes needs no scaling, so the pipeline has only the model step.

Numeric data (GaussianNB):

```
from sklearn.naive_bayes import GaussianNB

pipe = Pipeline([("model", GaussianNB())])

param_grid = {
    "model__var_smoothing": np.logspace(-11, -5, 7),   # 1e-11 ... 1e-5
}

gnb_grid = run_grid(pipe, param_grid)       # 7 combinations x 5 folds = 35
fits
```

Result on the breast cancer data: the default 1e-9 was best, 0.936 cross-validation accuracy, 0.939 test accuracy. Naive Bayes scores lower here than other models because these 30 features are strongly correlated, which breaks its independence assumption.

Text data (MultinomialNB): the vectorizer's settings can be tuned in the same grid. Here `texts` is a list of raw emails and `labels` their classes; with real data, split into train and test first.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

text_pipe = Pipeline([
    ("vectorizer", CountVectorizer()),
    ("model", MultinomialNB()),
])

param_grid = {
    "vectorizer__ngram_range": [(1, 1), (1, 2)],      # single words, or
words + word pairs
    "vectorizer__stop_words": [None, "english"],
    "model__alpha": [0.01, 0.1, 0.5, 1.0, 2.0],
    "model__fit_prior": [True, False],
}

text_grid = GridSearchCV(text_pipe, param_grid, cv=5, scoring="accuracy",
n_jobs=-1)
text_grid.fit(texts, labels)
print(text_grid.best_params_)
```

Support Vector Machine (SVC)

`C` and, for the RBF kernel, `gamma` are the two parameters to tune together. Large `C` punishes training mistakes heavily (narrow margin, overfitting risk); large `gamma` makes each point's influence very local (wiggly boundary, overfitting risk). Always scale features.

Parameter	Default	What it controls	Typical values to try
`C`	1.0	Price of each margin violation	0.1, 1, 10, 100
`kernel`	`"rbf"`	Shape of the boundary	`"linear"`, `"rbf"`, `"poly"`
`gamma`	`"scale"`	Reach of each point's influence (rbf, poly, sigmoid only)	`"scale"`, 0.001, 0.01, 0.1, 1
`degree`	3	Polynomial degree (poly only)	2, 3, 4
`class_weight`	None	Extra weight for rare classes	None, `"balanced"`
`probability`	False	Enables `predict_proba`; slower training	True only if you need probabilities

`gamma` has no effect on the linear kernel, so the grid below is a **list of two dictionaries**. Each dictionary is searched separately, which avoids wasted fits.

```
from sklearn.svm import SVC

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", SVC()),
])

param_grid = [
    {"model__kernel": ["linear"],
     "model__C": [0.1, 1, 10, 100]},
    {"model__kernel": ["rbf"],
     "model__C": [0.1, 1, 10, 100],
     "model__gamma": ["scale", 0.001, 0.01, 0.1, 1]},
]

svc_grid = run_grid(pipe, param_grid)      # 4 + 20 = 24 combinations x 5
folds = 120 fits
```

Result on the breast cancer data: best RBF kernel with `C=10` and `gamma=0.01`, 0.980 cross-validation accuracy, 0.982 test accuracy. For large datasets (100,000+ rows), `LinearSVC` trains much faster than `SVC(kernel="linear")`.

Decision Tree

An untuned tree keeps splitting until every leaf is pure, which almost always overfits. The parameters that matter most are the ones that stop growth early: `max_depth` and `min_samples_leaf`.

Parameter	Default	What it controls	Typical values to try
`max_depth`	None (no limit)	Maximum number of levels	None, 3, 5, 7, 10
`min_samples_split`	2	Fewest samples a node needs before it may split	2, 5, 10
`min_samples_leaf`	1	Fewest samples allowed in a leaf	1, 2, 5
`criterion`	`"gini"`	How split quality is measured	`"gini"`, `"entropy"`
`max_features`	None (all)	Features considered at each split	None, `"sqrt"`
`ccp_alpha`	0.0	Pruning strength after growth; higher = smaller tree	0.0, 0.001, 0.01
`class_weight`	None	Extra weight for rare classes	None, `"balanced"`

```
from sklearn.tree import DecisionTreeClassifier

pipe = Pipeline([("model", DecisionTreeClassifier(random_state=42))])

param_grid = {
    "model__criterion": ["gini", "entropy"],
    "model__max_depth": [None, 3, 5, 7, 10],
    "model__min_samples_split": [2, 5, 10],
    "model__min_samples_leaf": [1, 2, 5],
}

dt_grid = run_grid(pipe, param_grid)       # 90 combinations x 5 folds = 450
fits
```

Result on the breast cancer data: best Gini, `max_depth=5`, `min_samples_split=5`, 0.938 cross-validation accuracy, 0.921 test accuracy. `random_state` fixes the tie-breaking between equally good splits, so results repeat run to run.

Random Forest

A forest averages many deep trees, so it overfits far less than a single tree and works well with defaults. Tune `max_features` and `max_depth` first; more trees (`n_estimators`) rarely hurt accuracy, they only cost time.

Parameter	Default	What it controls	Typical values to try
`n_estimators`	100	Number of trees	100, 200, 300
`max_features`	`"sqrt"`	Features each split may consider; lower = more varied trees	`"sqrt"`, `"log2"`, 0.5
`max_depth`	None	Maximum depth of each tree	None, 5, 10
`min_samples_leaf`	1	Fewest samples in a leaf	1, 2, 4
`min_samples_split`	2	Fewest samples to split a node	2, 5, 10

Parameter	Default	What it controls	Typical values to try
`bootstrap`	True	Whether each tree trains on a random resample	True (needed for `oob_score`)
`class_weight`	None	Extra weight for rare classes	None, `"balanced"`, `"balanced_subsample"`
`n_jobs`	None	CPU cores used for training	-1 = all cores

```
from sklearn.ensemble import RandomForestClassifier

pipe = Pipeline([("model", RandomForestClassifier(random_state=42))])

param_grid = {
    "model__n_estimators": [100, 200, 300],
    "model__max_depth": [None, 5, 10],
    "model__max_features": ["sqrt", "log2"],
    "model__min_samples_leaf": [1, 2, 4],
}

rf_grid = run_grid(pipe, param_grid)       # 54 combinations x 5 folds = 270
fits
```

Result on the breast cancer data: best 200 trees, no depth limit, `max_features="sqrt"`, 0.960 cross-validation accuracy, 0.956 test accuracy. This grid took about 2 minutes on one CPU core, which is why large grids are better handled by `RandomizedSearchCV` (see Grid search tips).

Gradient Boosting

Boosting builds small trees one after another, each correcting the previous ones' errors. `learning_rate` and the number of trees work as a pair: a smaller learning rate needs more trees but usually generalizes better. scikit-learn has two classes:

- `GradientBoostingClassifier`: the classic version, fine for datasets up to about 10,000 rows.

- `HistGradientBoostingClassifier`: bins features into histograms first, so it is much faster on large data and handles missing values on its own. Its number of trees is `max_iter`, not `n_estimators`.

GradientBoostingClassifier parameters

Parameter	Default	What it controls	Typical values to try
`learning_rate`	0.1	How much each tree's correction counts	0.01, 0.1, 0.2
`n_estimators`	100	Number of boosting stages (trees)	100, 200, 500
`max_depth`	3	Depth of each small tree	2, 3, 4
`subsample`	1.0	Fraction of rows used per tree; below 1.0 adds randomness	0.8, 1.0
`min_samples_leaf`	1	Fewest samples in a leaf	1, 5, 10
`max_features`	None	Features considered per split	None, `"sqrt"`

```
from sklearn.ensemble import GradientBoostingClassifier

pipe = Pipeline([("model", GradientBoostingClassifier(random_state=42))])

param_grid = {
    "model__n_estimators": [100, 200],
    "model__learning_rate": [0.01, 0.1, 0.2],
    "model__max_depth": [2, 3, 4],
    "model__subsample": [0.8, 1.0],
}

gb_grid = run_grid(pipe, param_grid)       # 36 combinations x 5 folds = 180
fits
```

Result on the breast cancer data: best 200 trees, `learning_rate=0.2`, `max_depth=2`, `subsample=0.8`, 0.978 cross-validation accuracy, 0.956 test accuracy.

HistGradientBoostingClassifier parameters

Parameter	Default	What it controls	Typical values to try
`learning_rate`	0.1	Step size of each correction	0.05, 0.1, 0.2
`max_iter`	100	Number of trees	100, 200, 500
`max_leaf_nodes`	31	Leaves per tree (main size control)	15, 31, 63
`min_samples_leaf`	20	Fewest samples in a leaf	10, 20, 50
`l2_regularization`	0.0	Penalty on large leaf values	0.0, 1.0
`early_stopping`	`"auto"`	Stops adding trees when validation score stops improving; on automatically above 10,000 rows	`"auto"`

```
from sklearn.ensemble import HistGradientBoostingClassifier

pipe = Pipeline([("model", HistGradientBoostingClassifier(random_state=42))])

param_grid = {
    "model__learning_rate": [0.05, 0.1, 0.2],
    "model__max_iter": [100, 200],
    "model__max_leaf_nodes": [15, 31],
    "model__min_samples_leaf": [10, 20],
    "model__l2_regularization": [0.0, 1.0],
}

hgb_grid = run_grid(pipe, param_grid)      # 48 combinations x 5 folds = 240
fits
```

Result on the breast cancer data: best `learning_rate=0.1`, 200 trees, 31 leaves, `min_samples_leaf=10`, 0.978 cross-validation accuracy, 0.956 test accuracy, in about half the time of the classic version.

AdaBoost

AdaBoost trains weak learners one after another, giving more weight to the samples the previous learners got wrong. Tune `n_estimators` and `learning_rate` together, and the depth of the weak learner through the nested `estimator` parameter.

Parameter	Default	What it controls	Typical values to try
`estimator`	None (a depth-1 decision tree, a "stump")	The weak learner that is boosted	`DecisionTreeClassifier(max_depth=1)`
`n_estimators`	50	Number of weak learners	50, 100, 200
`learning_rate`	1.0	How much each learner's vote counts	0.01, 0.1, 0.5, 1.0
`estimator__max_depth`	1	Depth of each weak learner (nested parameter)	1, 2, 3

Nested parameters: the tree inside AdaBoost is itself a parameter, so its settings take two double underscores: `model__estimator__max_depth` means "the `max_depth` of the `estimator` of the step named `model`".

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

pipe = Pipeline([
    ("model", AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=1),
        random_state=42)),
])

param_grid = {
    "model__n_estimators": [50, 100, 200],
    "model__learning_rate": [0.01, 0.1, 0.5, 1.0],
    "model__estimator__max_depth": [1, 2, 3],
}

ada_grid = run_grid(pipe, param_grid)      # 36 combinations x 5 folds = 180
fits
```

Result on the breast cancer data: best 100 stumps (depth 1), `learning_rate=0.5`, 0.978 cross-validation accuracy, 0.956 test accuracy.

Version note: older tutorials use `base_estimator` and `algorithm="SAMME.R"`. Both were removed in recent releases; use `estimator` and leave `algorithm` out.

Grid search tips

Choose the right score. Accuracy is misleading when classes are imbalanced (for example 95% "not fraud"). Set `scoring` to match the goal:

`scoring` value	Use it when
`"accuracy"`	Classes are roughly balanced
`"f1"`	Binary problem with an imbalanced positive class
`"f1_macro"`	Multiclass, every class equally important
`"recall"`	Missing a positive is costly (disease, fraud)
`"roc_auc"`	You care about ranking, not one threshold

Use RandomizedSearchCV for large grids. Grid search tries every combination, so 5 parameters with 5 values each is 3,125 combinations. `RandomizedSearchCV` tries a fixed number of random combinations (`n_iter`) and can sample from ranges instead of lists:

```
from scipy.stats import loguniform
from sklearn.model_selection import RandomizedSearchCV
from sklearn.svm import SVC

pipe = Pipeline([("scaler", StandardScaler()), ("model", SVC())])

param_dist = {
    "model__C": loguniform(0.01, 1000),      # any value from 0.01 to 1000
    "model__gamma": loguniform(0.0001, 1),
}

search = RandomizedSearchCV(pipe, param_dist, n_iter=30, cv=5,
                            scoring="f1", n_jobs=-1, random_state=42)
search.fit(X_train, y_train)               # 30 combinations x 5 folds = 150
fits
print(search.best_params_, search.best_score_)
```

`loguniform` spreads samples evenly across orders of magnitude, which suits parameters like `C`, `gamma` and `alpha`.

Look at all results, not only the best. If several settings score almost the same, prefer the simpler model.

```
import pandas as pd

results = pd.DataFrame(search.cv_results_)
results[["params", "mean_test_score", "std_test_score", "rank_test_score"]] \
    .sort_values("rank_test_score").head(10)
```

Common mistakes:

1. **Scaling before splitting.** Fitting `StandardScaler` on all the data leaks test information. Put the scaler inside the pipeline, as every snippet here does.
2. **Tuning on the test set.** `GridSearchCV` uses only the training data for its folds. Touch the test set once, at the end.
3. **The best value is at the edge of the grid.** If the best `C` is 100 and 100 is your largest value, extend the grid (for example to 1000) and search again.
4. **Grids that are too fine.** Start coarse with values that differ by a factor of 10 (0.01, 0.1, 1, 10), then search more finely around the best one.
5. **Forgetting `random_state`.** Without it, trees, forests and the train/test split change on every run, so results cannot be compared.

Practice exercises

Use the setup code and snippets above. Write down your numbers before answering each question.

1. **Scaling matters for KNN.** Remove the `StandardScaler` step from the KNN pipeline and run the grid again. How much does the test accuracy change? Why does KNN care about scale when a decision tree does not?
2. **Edge of the grid.** In the Logistic Regression grid, add 1000 to the list of `C` values. Does the best `C` change? What would you do if the best value were 1000?
3. **Pruning a tree.** Add `"model__ccp_alpha": [0.0, 0.001, 0.01]` to the Decision Tree grid. After fitting, check the depth of the best tree with `dt_grid.best_estimator_["model"].get_depth()`. Is it smaller than before?

4. **A different score.** Change `scoring="accuracy"` to `scoring="f1"` in `run_grid` and rerun the SVM grid. Do the best parameters change?
5. **Random instead of grid.** Rewrite the Random Forest search with `RandomizedSearchCV` and `n_iter=15`. Compare the time taken and the best score with the full grid of 54 combinations.
6. **Compare all models.** Make a table of the best cross-validation accuracy and test accuracy for every algorithm. Which model would you choose, and why might the test accuracy differ from the cross-validation accuracy?